RAG: поиск и индексы

01 Лексический поиск: TF-IDF, BM25

Терм - поисковая единица после анализа текста: слово, основа или сохранённый код. Это не обязательно токен LLM. **TF** - число вхождений терма в документ; **IDF** - вес его редкости в корпусе, коллекции документов. **TF-IDF** объединяет частоту внутри документа и редкость между документами.

Мини-корпус из трёх документов: A: «возврат возврат деньги»; B: «возврат срок»; C: «err-4031 доступ». Используем учебное соглашение IDF(t)=ln(N/df(t)): t - терм, N=3 - число документов, df - сколько документов содержат терм, ln - натуральный логарифм.

«Возврат» есть в A и B: df=2, IDF=ln(3/2)≈0,405. «Деньги» и «err-4031» встречаются в одном документе: df=1, IDF=ln(3)≈1,099.

Вес «возврат» в A: TF×IDF=2×0,405=0,810; в B: 1×0,405=0,405. Код в C: 1×1,099=1,099. Терм во всех документах получил бы ln(1)=0. Реальные реализации могут сглаживать и нормализовать иначе.

BM25 ограничивает выигрыш от повторов и учитывает длину. При k1=1,2, где k1 управляет насыщением, частотный множитель без поправки длины: TF×(k1+1)/(TF+k1).

TF=1: 2,2/2,2=1. TF=2: 4,4/3,2=1,375. TF=10: 22/11,2≈1,964. Десять повторов не дают десятикратной пользы. Это множитель, не полный score.

Поправка заменяет k1 в знаменателе на k1×(1−b+b×dl/avgdl): b - сила поправки; dl - длина документа в термах; avgdl - средняя длина. При b=0 поправки нет. BM25 не создаёт совпадение синонимов без общего терма или дополнительной обработки.

02 Sparse, dense и фильтры

Sparse, разреженное представление: координат много, большинство у конкретного текста нулевые. В словарном варианте координата соответствует терму. **Dense**, плотное представление: фиксированный вектор, обычно с преимущественно ненулевыми координатами. Обучаемый sparse тоже возможен; sparse не равен исключительно BM25.

Инвертированный индекс хранит для терма список документов. По «возврат» сразу смотрим A и B. Dense-поиск кодирует вопрос и ищет близкие векторы в совместимом пространстве. Для точного ERR-4031 полезен лексический сигнал; для «деньги назад» - семантический. Ни один сам не гарантирует правильное правило.

Векторное хранилище связывает эмбеддинги с ID и метаданными. **Фильтр** ограничивает кандидатов по организации, доступу, языку, статусу, версии и дате действия. Для текущего вопроса архив «30 дней» не должен конкурировать с действующим «45 дней». Исторический вопрос требует другого условия применимости.

Права проверяют до раскрытия: запрещённый текст не должен попасть пользователю, внешнему реранкеру, LLM, общему кэшу и обычным логам. Область доступа выводит доверенный сервер, а не произвольный team_id браузера. Удалить ссылку после передачи текста модели уже недостаточно.

Если фильтровать после ограниченного top-5 и четыре результата запрещены, останется один, хотя дальше могли быть полезные разрешённые. Нужны согласованные поиск, фильтрация и пополнение кандидатов. Для сравнения exact и ANN разрешённый корпус должен совпадать.

Ключ кэша учитывает доступ и версию. Метаданные тоже могут раскрывать внутреннюю информацию. Одинаковый сервер может поддерживать лексический и векторный поиск: две стратегии не обязательно требуют двух отдельных систем.

03 Exact, ANN, HNSW и IVF

Exact сравнивает запрос со всеми допустимыми векторами и выбирает top-k, лучшие k объектов по метрике. **ANN**, approximate nearest neighbors, сокращает работу, допуская пропуск точных соседей. Точный геометрический сосед ещё не обязательно полезный документ.

ANN recall@k - доля exact top-k, попавшая в ANN top-k. Exact: [A,B,C,D,E]; ANN: [A,B,C,F,G]. При k=5 общих три: ANN recall@5=3/5=0,6. Это точность приближения индекса; Retrieval Recall сравнивает с экспертной полезностью. Exact может иметь ANN recall=1 и бесполезную выдачу.

HNSW - многоуровневый граф: вершины - векторы, связи ведут к соседям. Верхние уровни помогают найти перспективную область, нижние уточняют поиск. Три настройки отвечают за разные работы.

M - связанность графа: больше связей обычно требует больше памяти и работы построения. **efConstruction** - ширина исследования при построении; более тщательная сборка дороже. **efSearch** - ширина исследования при запросе; рост обычно улучшает полноту ценой задержки. Top-k при этом можно оставить равным 5. Изменение M и efConstruction обычно требует перестроения; детали зависят от движка.

IVF группирует пространство вокруг центров. **nlist** - число групп, **nprobe** - сколько групп просматривают на запрос. Из 10 000 векторов в 100 примерно равных группах nprobe=1 даёт около 100 проверок, nprobe=10 - около 1000, плюс выбор центров.

Больше nlist при прежнем nprobe не гарантирует полноту: просматривается меньшая доля корпуса. Все группы с точными несжатыми векторами дают полный перебор; квантование добавляет отдельную ошибку. Сравнивайте качество и задержку на одном наборе; рост ширины не исправляет несовместимые эмбеддинги.

RAG: поиск и индексы

04 Гибридный поиск и RRF

Гибридный поиск объединяет, например, BM25 и dense. Складывать BM25=12 и cosine=0,8 без обоснования нельзя: разные шкалы. **RRF, reciprocal rank fusion**, использует позиции, а не сырые scores. Позиции начинаются с 1; у фрагмента должен быть стабильный ID.

RRF(d) = сумма 1/(c + rank) по спискам, содержащим d. Здесь d - фрагмент; rank - позиция в конкретном списке; c - константа сглаживания. Возьмём c=60. Отсутствующий фрагмент даёт вклад **0**, а не rank=0.

Позиция	BM25	Dense
1	A	B
2	B	C
3	D	A

A: есть на местах 1 и 3. RRF(A)=1/61+1/63≈0,016393+0,015873=0,032266.

B: места 2 и 1. RRF(B)=1/62+1/61≈0,016129+0,016393=0,032522.

C: отсутствует в BM25, второе место в dense. RRF(C)=0+1/62≈0,016129.

D: третье место в BM25, отсутствует в dense. RRF(D)=1/63+0≈0,015873.

Общий порядок: B, A, C, D. В чуть выше A благодаря сочетанию позиций. Это порядок объединения, а не доказательство истинности или вероятности правильного ответа. Если вместо нулевого вклада добавить 1/60, отсутствие ошибочно станет преимуществом.

Дедупликация: A из двух списков становится одной записью с двумя вкладами. Перекрывающиеся чанки с разными ID требуют отдельной обработки близких дублей. Нельзя выбрасывать единственный чанк с исключением только из-за похожести.

с сглаживает преимущество первых мест; 60 - соглашение примера. Глубина входных списков влияет на результат: фрагмент за границей списка не получает вклад. Качество гибрида подтверждают отдельными группами вопросов, включая коды и перефразировки.

05 Реранкер и бюджет контекста

Bi-encoder кодирует вопрос и документ отдельно. Векторы документов готовят заранее, на запросе считают один вектор вопроса. Это быстрый широкий отбор; тонкое условие может потеряться в сжатом представлении.

Cross-encoder обрабатывает пару «вопрос + фрагмент» совместно и оценивает соответствие. Это реранкер: замечает взаимодействие точного кода, отрицания и условий, но требует обработки каждой пары. Для 50 кандидатов - 50 пар; фактическая стоимость зависит от модели, длины и пакетной обработки.

Потолок кандидатов: реранкер переставляет только полученное. Если важное условие стоит на месте 25, а вход ограничен первыми 20, более точная модель его не восстановит. Сначала проверяют полноту отбора, затем первые позиции.

Разделяйте три числа: получить 40 кандидатов → переранжировать 30 после фильтров и дедупликации → отправить 5 в контекст. Поиск шире не обязан увеличивать запрос генератору, но влияет на задержку и стоимость.

Считаем бюджет. Окно модели - 8000 токенов. Вопрос и инструкции - 1000. Резерв ответа - 1500. Под документы остаётся **8000−1000−1500=5500**.

Тело чанка - 900 токенов; заголовок, ID и ссылка - ещё 50. Один готовый блок занимает **950**.

Шесть блоков: 6×950=5700, превышение на 200. Пять: 5×950=4750, остаётся 750. Значит в этих условиях помещаются **пять**, а не шесть. У настоящих чанков длины различаются: считают окончательно собранный запрос.

Бюджет расходуют повторы и противоречивые редакции. Сохраняйте правило с исключением, а не просто самые похожие строки. Если основания не помещаются, выбирайте стратегию отбора или уточняйте вопрос; молчаливое усечение условия способно изменить ответ.

06 Диагностика поисковой цепочки

Начните с покрытия: нужный документ опубликован, проиндексирован, действует и разрешён пользователю? Если инструкции ERR-4031 в доступном корпусе нет, настройка графа её не создаст. Правильное ограничение доступа нельзя считать поводом раскрыть закрытый документ.

Лексический путь не находит код. Проверьте анализатор: сохранил ли ERR-4031 целиком, не превратил ли в общий 403? Сравните точный запрос с перефразировкой. Отсутствие общего терма не объясняется параметрами HNSW.

Dense exact плох по смыслу. Проверьте совместимость модели, версии, префиксов, нормализации; затем чанки и соответствие типа поиска задаче. Exact точно воспроизводит геометрию, даже если она неудачна для нужного кода.

Exact хорош, ANN теряет соседа. При одинаковых векторах и фильтрах измерьте ANN recall и задержку. Для HNSW меняйте efSearch; для IVF - nprobe. Это разные индексы: нельзя автоматически «поднять оба параметра» у любой системы.

Кандидат найден, но исчез позже. Сохраните его ID и позиции: исходная выдача → RRF → вход реранкера → финальный порядок → контекст. Проверьте отсечение списков, объединение ID, дедупликацию, порядок и токены. Если ERR-4031 первый после реранкера, но отсутствует в промпте, подтверждённый разрыв находится в сборке контекста.

Контекст полный, ответ неверен. Сравните утверждения с источником: дата, число, условие, отрицание и ссылка. Тогда проверяют генерацию, а не автоматически перестраивают индекс.

Для эксперимента фиксируйте корпус, права, вопросы и бюджет. Меняйте один участок, заранее запишите ожидаемый результат и измерьте качество вместе со стоимостью. Средний выигрыш может скрыть полный провал на редких критичных кодах.